

COUDE

Escola Técnica de Programação e Robótica

Documentação Técnica de Infraestrutura

Servidor Centralizado de Arquivos, Autenticação e API de Integração

Versão: 2.0 (extensão profissional)

Domínio: ad.coude.com.br

Servidor: srv1.ad.coude.com.br | Debian 12 | Samba AD

Abril 2026

v2.0 inclui: Sistema de Usernames · API de Cadastro · Modelo RBAC · Melhorias de Engenharia

1. Visão Geral do Projeto

A COUDE é uma escola técnica credenciada pelo MEC que oferece cursos presenciais de Programação Full Stack, Banco de Dados e Robótica. O ambiente de laboratório conta com aproximadamente 30 computadores Windows 10 compartilhados por cerca de 300 alunos, mais de 30 turmas, professores e monitores.

Problemas que este projeto resolve:

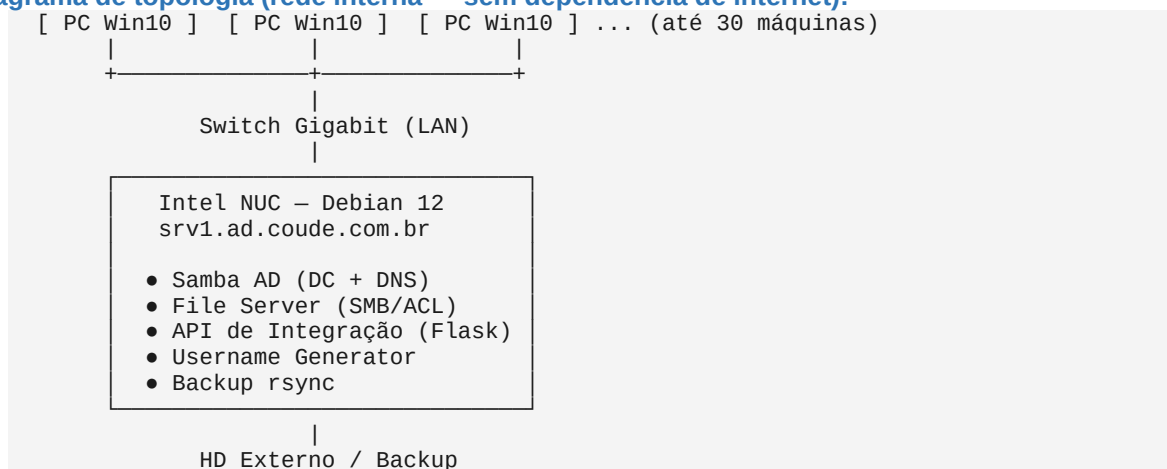
- Exclusão ou alteração acidental de arquivos de outros usuários
- Ausência de separação de dados por usuário, turma e cargo
- Falta de backup centralizado e rastreabilidade de alterações
- Perda de dados quando uma máquina é formatada ou trocada

1.1 Solução Adotada

Um único servidor Debian 12 atuando simultaneamente como Domain Controller (DC) e File Server (FS) usando Samba como Active Directory. Os computadores Windows 10 ingressam no domínio e têm suas pastas pessoais redirecionadas de forma transparente para o servidor — o usuário acredita estar usando o disco local enquanto os arquivos são centralizados no servidor.

1.2 Topologia Geral

Diagrama de topologia (rede interna — sem dependência de internet):



NOTA: Servidor DEVE estar em cabo Gigabit. Clientes podem usar Wi-Fi.

2. Decisões Técnicas

2.1 Por que Samba como Active Directory

Compatibilidade nativa com Windows 10, suporte a GPO, autenticação centralizada via Kerberos, controle de permissões via ACL NTFS e DNS interno integrado — sem custo de licença. A alternativa Samba + LDAP puro foi descartada por ausência de GPO nativo e manutenção mais complexa.

2.2 Por que ad.coude.com.br

Usar coude.com.br diretamente no AD causaria conflito de DNS com o site público da escola. Usar coude.local tem risco de conflito com extensões .local em redes mDNS. A solução é o subdomínio interno dedicado ad.coude.com.br: separa completamente o AD do domínio público sem abrir mão da identidade institucional.

2.3 Um servidor: DC + FS + API

Para o porte atual da COUDE (30 PCs, ~300 alunos), consolidar DC, FS e API no mesmo servidor é a decisão mais adequada. A separação em múltiplos servidores só se justifica acima de 500 requisições simultâneas de alta intensidade. O servidor único reduz custo, complexidade operacional e superfície de falha.

 **ATENÇÃO:** Se o servidor cair, domínio, arquivos e API ficam indisponíveis. UPS (nobreak) é obrigatório em produção.

3. Nomenclatura Padrão

3.1 Infraestrutura

Item	Nome
Domínio AD	ad.coude.com.br
Servidor (FQDN)	srv1.ad.coude.com.br
IP do servidor	192.168.1.10 (fixo — ajustar à rede real)
Share usuários	\\srv1\users
Share turmas	\\srv1\turmas
Share professores	\\srv1\professores
Share monitores	\\srv1\monitores

3.2 Usernames — Sistema v2 (ver Seção 16)

O sistema anterior de nomes como aluno.2026.001 foi substituído por um algoritmo de geração determinística baseado no nome completo da pessoa. A especificação completa, pseudocódigo, implementação PHP/JS e estratégia de colisão estão na Seção 16.

NOTA: Usernames são gerados pelo servidor no momento do cadastro. O sistema externo envia apenas o nome completo e recebe o username gerado. Ver Seção 17 para a API de integração.

Cargo	Padrão de username	Exemplo gerado	Lógica
Aluno	Gerado por algoritmo	thiago.cer	Combinação nome+sobrenome (ver S.16)
Professor	Gerado por algoritmo	ana.lima	Mesma lógica, prefixo de cargo opcional
Monitor	Gerado por algoritmo	lucas.oliv	Mesma lógica
Admin	Gerado por algoritmo	dani.sil	Mesmo algoritmo, OU separada

3.3 Grupos

Tipo	Exemplos
Grupos de cargo	alunos, professores, monitores, administradores
Grupos de turma	turma_fullstack_001, turma_banco_001, turma_robotica_001

3.4 OUs no Active Directory

Árvore de OUs:

```

ad.coude.com.br
├── OU=Usuarios
│   ├── OU=Alunos
│   └── OU=Professores

```

```
├── OU=Monitores
├── OU=Administradores
├── OU=Turmas
├── OU=Computadores
└── OU=Deletados ← contas em período de soft delete (30 dias)
```

4. Instalação do Debian 12

4.1 Hardware Recomendado

Componente	Especificação
CPU	Intel NUC i5/i7 — 4+ núcleos
RAM	16 GB (mínimo 8 GB)
Disco sistema	SSD M.2 NVMe — 256 GB
Disco dados	SSD/HD 2.5" — 2 TB (arquivos dos usuários)
Rede	Gigabit Ethernet (cabo ao switch, obrigatório)
Energia	UPS/nobreak obrigatório em produção

4.2 Opções na Instalação

1. Hostname: srv1 | Domain: ad.coude.com.br
2. Particionamento: "Separate /var and /srv, swap < 1GB (for servers)" — isola logs e dados de serviços
3. Software: marcar apenas SSH server + standard system utilities — ZERO interface gráfica

⚠️ ATENÇÃO: Não instalar ambiente gráfico. Servidor headless. Administração 100% via SSH.

4.3 Pós-Instalação

Hostname, /etc/hosts e IP fixo:

```
hostnamectl set-hostname srv1.ad.coude.com.br

# /etc/hosts — OBRIGATÓRIO usar IP real, nunca 127.x
127.0.0.1      localhost
192.168.1.10   srv1.ad.coude.com.br srv1

# IP fixo em /etc/network/interfaces
auto enp0s3
iface enp0s3 inet static
    address 192.168.1.10
    netmask 255.255.255.0
    gateway 192.168.1.1
    dns-nameservers 127.0.0.1

systemctl restart networking

# Validar
hostname -f # deve retornar: srv1.ad.coude.com.br
```

✖ ERRO FATAL: O hostname do DC nunca pode resolver para 127.x.x.x. Se apontar para loopback o domínio nasce quebrado e o Windows não consegue ingressar.

5. Samba AD — Instalação e Configuração

5.1 Instalar Pacotes

Instalar dependências:

```
apt update && apt upgrade -y
apt install -y samba krb5-user winbind smbclient dnsutils acl attr python3-samba
# Realm quando perguntado: AD.COUDE.COM.BR (maiúsculas)
```

5.2 Provisionar Domínio

Provisionar — responder cada campo conforme abaixo:

```
mv /etc/samba/smb.conf /etc/samba/smb.conf.bak
samba-tool domain provision --use-rfc2307 --interactive

# Realm:          AD.COUDE.COM.BR
# Domain:         COUDE
# Server Role:    dc
# DNS backend:    SAMBA_INTERNAL
# DNS forwarder:  8.8.8.8
# Admin password: (senha forte, mínimo 12 chars, maiúsc+minúsc+núm+símbolo)
```

5.3 Ativar Serviço

Desabilitar serviços conflitantes e ativar o DC:

```
systemctl stop smbd nmbd winbind
systemctl disable smbd nmbd winbind
systemctl unmask samba-ad-dc
systemctl enable samba-ad-dc
systemctl start samba-ad-dc

# Proteger resolv.conf
cat > /etc/resolv.conf <<EOF
nameserver 127.0.0.1
search ad.coude.com.br
EOF
chattr +i /etc/resolv.conf
```

5.4 Validação

Todos devem passar antes de continuar:

```
samba-tool domain level show
host -t A srv1.ad.coude.com.br
host -t SRV _ldap._tcp.ad.coude.com.br
kinit Administrator@AD.COUDE.COM.BR
smbclient -L srv1 -U Administrator
```

✓ **BOA PRÁTICA:** Se todos passarem, o DC está operacional. Só continuar após confirmar cada um.

5.5 Criar OUs e Grupos Base

OUs e grupos:

```
# OUs
```

```
samba-tool ou create "OU=Usuarios,DC=ad,DC=coude,DC=com,DC=br"
samba-tool ou create "OU=Alunos,OU=Usuarios,DC=ad,DC=coude,DC=com,DC=br"
samba-tool ou create "OU=Professores,OU=Usuarios,DC=ad,DC=coude,DC=com,DC=br"
samba-tool ou create "OU=Monitores,OU=Usuarios,DC=ad,DC=coude,DC=com,DC=br"
samba-tool ou create "OU=Administradores,OU=Usuarios,DC=ad,DC=coude,DC=com,DC=br"
samba-tool ou create "OU=Turmas,DC=ad,DC=coude,DC=com,DC=br"
samba-tool ou create "OU=Computadores,DC=ad,DC=coude,DC=com,DC=br"
samba-tool ou create "OU=Deletados,DC=ad,DC=coude,DC=com,DC=br"

# Grupos de cargo
samba-tool group add alunos
samba-tool group add professores
samba-tool group add monitores
samba-tool group add administradores
```

6. Estrutura de Pastas no Servidor

Esta seção define a estrutura de diretórios base. O modelo completo de permissões e controle de acesso (RBAC) está na Seção 18, que é a referência definitiva.

Criar estrutura de diretórios:

```
mkdir -p /srv/samba/{users,turmas,professores,monitores,arquivo_morto}

# Habilitar ACL POSIX (necessário para RBAC granular)
# Verificar se acl está nas opções de mount:
mount | grep "/srv"
# Se não aparecer "acl", adicionar ao /etc/fstab:
# UUID=... /srv ext4 defaults,acl 0 2

# Ativar sem reboot:
mount -o remount,acl /srv
```

6.1 Árvore Completa de Armazenamento

Estrutura por turma e usuário (ver Seção 18 para permissões):

```
/srv/samba/
├── users/
│   ├── thiago.cer/      ← pasta pessoal do aluno
│   │   ├── Documents/
│   │   ├── Desktop/
│   │   └── Downloads/
│   └── ana.lima/        ← pasta pessoal da professora
├── turmas/
│   └── turma_fullstack_001/
│       ├── _geral/      ← pasta raiz da turma (professores full, alunos read)
│       │   ├── Materiais/
│       │   └── Avisos/
│       └── alunos/      ← subpastas individuais
│           ├── thiago.cer/
│           │   ├── Projetos/
│           │   └── Entregas/
│           └── joao.silv/
├── professores/
│   └── ana.lima/        ← área privada da professora
├── monitores/
└── arquivo_morto/      ← dados de contas expiradas (30d)
```

7. Configuração do Samba (smb.conf)

Arquivo completo /etc/samba/smb.conf:

```
# =====
# COUDE – Samba AD + File Server v2.0
# =====
[global]
    workgroup = COUDE
    realm = AD.COUDE.COM.BR
    netbios name = SRV1
    server role = active directory domain controller
    dns forwarder = 8.8.8.8
    log file = /var/log/samba/samba.log
    max log size = 1000
    log level = 1
    vfs objects = acl_xattr
    map acl inherit = yes
    store dos attributes = yes

[sysvol]
    path = /var/lib/samba/sysvol
    read only = No

[netlogon]
    path = /var/lib/samba/sysvol/ad.coude.com.br/scripts
    read only = No

[users]
    comment = Pastas Pessoais dos Usuarios
    path = /srv/samba/users
    read only = No
    browseable = No
    valid users = @alunos @administradores @professores @monitores
    create mask = 0600
    directory mask = 0700
    vfs objects = acl_xattr
    map acl inherit = yes

[turmas]
    comment = Pastas das Turmas
    path = /srv/samba/turmas
    read only = No
    browseable = Yes
    valid users = @alunos @professores @monitores @administradores
    create mask = 0660
    directory mask = 0770
    vfs objects = acl_xattr
    map acl inherit = yes

[professores]
    comment = Area Privada dos Professores
    path = /srv/samba/professores
    read only = No
    browseable = No
    valid users = @professores @administradores
    create mask = 0660
    directory mask = 0770
    vfs objects = acl_xattr
    map acl inherit = yes

[monitores]
    comment = Area dos Monitores
    path = /srv/samba/monitores
    read only = No
    browseable = No
    valid users = @monitores @administradores
    create mask = 0660
    directory mask = 0770
```

```
vfs objects = acl_xattr  
map acl inherit = yes
```

Reiniciar e validar:

```
systemctl restart samba-ad-dc  
smbclient -L srv1 -U Administrator # deve listar todos os shares
```

8. Scripts de Gerenciamento (Legacy — ver Seção 17 para API)

Os scripts abaixo são a camada de execução chamada pela API de integração (Seção 17). Eles também podem ser executados diretamente pelo administrador no servidor via SSH.

8.1 Criar Pasta de Usuário

/usr/local/bin/coude-create-user-dir.sh

```
#!/bin/bash
# Uso: coude-create-user-dir.sh <username> <cargo>
USERNAME=$1; CARGO=$2
case "$CARGO" in
    aluno)   BASEDIR="/srv/samba/users" ;;
    prof)    BASEDIR="/srv/samba/professores" ;;
    monitor) BASEDIR="/srv/samba/monitores" ;;
    admin)   BASEDIR="/srv/samba/users" ;;
    *)       echo "Cargo inválido"; exit 1 ;;
esac
USERDIR="$BASEDIR/$USERNAME"
mkdir -p "$USERDIR"/{Documents,Desktop,Downloads}
chown -R root:root "$USERDIR"
chmod -R 700 "$USERDIR"
setfacl -R -m u:"$USERNAME":rwx "$USERDIR"
setfacl -R -d -m u:"$USERNAME":rwx "$USERDIR"
setfacl -R -m g:"Domain Admins":rwx "$USERDIR"
setfacl -R -d -m g:"Domain Admins":rwx "$USERDIR"
echo "Pasta criada: $USERDIR"
```

8.2 Soft Delete (30 dias)

/usr/local/bin/coude-soft-delete.sh

```
#!/bin/bash
USERNAME=$1
LOGFILE="/var/log/coude/soft_delete.log"
DATA=$(date +%Y-%m-%d)
samba-tool user disable "$USERNAME"
samba-tool user move "$USERNAME" "OU=Deletados,DC=ad,DC=coude,DC=com,DC=br"
samba-tool group removemembers alunos "$USERNAME" 2>/dev/null
echo "$DATA|$USERNAME|SOFT_DELETE" >> "$LOGFILE"
echo "Conta $USERNAME desativada. Purge em 30 dias."
```

8.3 Purge de Contas Expiradas

/usr/local/bin/coude-purge-deleted.sh

```
#!/bin/bash
LOGFILE="/var/log/coude/soft_delete.log"
ARCHIVE="/srv/samba/arquivo_morto"
PURGE_DAYS=30
while IFS="|" read -r DATA USERNAME ACAO; do
    [ "$ACAO" != "SOFT_DELETE" ] && continue
    DATA_EPOCH=$(date -d "$DATA" +%s 2>/dev/null)
    DIFF_DAYS=$(( (date +%s) - DATA_EPOCH ) / 86400 )
    if [ "$DIFF_DAYS" -ge "$PURGE_DAYS" ]; then
        for DIR in /srv/samba/users /srv/samba/professores /srv/samba/monitores; do
            [ -d "$DIR/$USERNAME" ] && mv "$DIR/$USERNAME" "$ARCHIVE/${USERNAME}_${DATA}"
        done
        samba-tool user delete "$USERNAME" 2>/dev/null
    fi
done
```

```
    echo "$(date +%Y-%m-%d)|$USERNAME|PURGED" >> "$LOGFILE"
  fi
done < "$LOGFILE"
```

9–15. API Flask, GPO, Cron, Firewall, Checklist, Riscos e Quick Reference

i **NOTA:** As seções 9 a 15 da v1.0 (API Flask original, GPO/Windows, Cron Jobs, Firewall UFW, Checklist, Riscos e Quick Reference) permanecem válidas. A API de integração foi completamente redesenhada e expandida na Seção 17. Consulte o documento v1.0 ou as seções 17 em diante para a especificação atualizada.

Resumo do que não mudou nas seções 9–15

- GPO de redirecionamento de pastas (Seção 10): sem alteração
- Firewall UFW com todas as portas do Samba AD (Seção 12): sem alteração
- Cron jobs de purge e backup (Seção 11): sem alteração
- Checklist de implantação (Seção 13): sem alteração
- Tabela de riscos e escalabilidade (Seção 14): complementada na Seção 19
- Quick Reference de comandos (Seção 15): complementada na Seção 19

Firewall (resumo obrigatório)

Portas essenciais Samba AD + API:

```
ufw allow 22/tcp      # SSH
ufw allow 53          # DNS
ufw allow 88          # Kerberos
ufw allow 389         # LDAP
ufw allow 636/tcp     # LDAPS
ufw allow 135/tcp     # RPC Endpoint Mapper
ufw allow 445/tcp     # SMB
ufw allow 49152:65535/tcp # RPC dynamic ports
ufw allow 123/udp     # NTP
# API: somente rede interna
ufw allow from 192.168.1.0/24 to any port 8080
ufw enable
```

16. Sistema de Geração de Usernames (v2 — Substituição Completa)

Esta seção substitui completamente o padrão anterior (aluno.2026.001, prof.thiago, etc). O novo sistema gera usernames de forma determinística a partir do nome completo da pessoa, garantindo unicidade sem uso de números ou símbolos.

16.1 Requisitos do Sistema

Requisito	Especificação
Comprimento	4 a 8 caracteres
Caracteres	Apenas letras a-z minúsculas (sem números, sem ponto, sem hífen)
Composição	Fragmento do primeiro nome + fragmento do sobrenome principal
Tamanho mínimo	Mínimo 3 caracteres de cada parte (nome e sobrenome)
Unicidade	Garantida por estratégia determinística de variações sem sufixos numéricos
Geração	100% server-side — o sistema externo nunca define o username
Ordenação	Resultados mais legíveis primeiro, mais complexos depois

16.2 Abordagem Técnica: Geração Substrings Combinatorial

A técnica é chamada Combinatorial Substring Generation com Collision-Free Deterministic Fallback. O algoritmo funciona em duas fases:

Fase 1 — Normalização do nome

- Converter tudo para lowercase
- Remover acentos e diacríticos (ã → a, ç → c, é → e etc.)
- Remover partículas irrelevantes: de, da, dos, das, do, e
- Separar em tokens: primeiro nome e sobrenomes significativos

Fase 2 — Geração do espaço de candidatos

O algoritmo gera um array ordenado de candidatos a username usando o produto combinatório de substrings do primeiro nome (3..N) × substrings do último sobrenome (3..M), priorizando combinações mais curtas e mais legíveis primeiro:

Pseudocódigo: geração do espaço de candidatos

```
function generateCandidates(firstName, lastName):
    candidates = []

    # Substrings do primeiro nome: do início, tamanho 3 até len(firstName)
    nameFragments = [firstName[0..k] for k in 3..len(firstName)]
    # → "thiago" → ["thi", "thia", "thiag", "thiago"]
```

```
# Substrings do sobrenome: do início, tamanho 3 até min(5, len(lastName))
surnameFrgs = [lastName[0..k] for k in 3..min(5, len(lastName))]
# → "cerqueira" → ["cer", "cerq", "cerqu"]

# Produto combinatório, ordenado por comprimento total crescente
for each (nf, sf) in cartesianProduct(nameFrgs, surnameFrgs):
    candidate = nf + sf          # ex: "thicer", "thicerq"
    if len(candidate) >= 4 and len(candidate) <= 8:
        candidates.append(candidate)

# Variação alternativa: sobrenome primeiro
for each (sf, nf) in cartesianProduct(surnameFrgs, nameFrgs):
    candidate = sf + nf          # ex: "certhi", "cerqthi"
    if len(candidate) >= 4 and len(candidate) <= 8:
        candidates.append(candidate)

# Remover duplicatas, manter ordem
return unique(candidates)
```

16.3 Estratégia de Colisão: Zero Números, 100% Determinístico

Se o primeiro candidato já existe, o sistema avança para o próximo no array pré-gerado. Nunca usa números. O espaço de candidatos para um nome típico tem entre 15 e 40 variações possíveis — suficiente para centenas de usuários com o mesmo nome.

Pseudocódigo: seleção com colisão determinística

```
function resolveUsername(fullName, existingUsernames):
    (firstName, lastName) = parseName(fullName)
    candidates = generateCandidates(firstName, lastName)

    for candidate in candidates:
        if candidate NOT IN existingUsernames:
            return candidate

    # Fallback extremo: usar segundo sobrenome como fonte de variação
    secondName = getSecondSurname(fullName)
    extraCandidates = generateCandidates(firstName, secondName)
    for candidate in extraCandidates:
        if candidate NOT IN existingUsernames:
            return candidate

    # Se ainda assim colidir (estatisticamente improvável com <10000 users):
    raise UsernameExhaustedException(fullName)
```

⚠ ATENÇÃO: O sistema NÃO usa números ou sufixos. Se o espaço de candidatos se esgotar (improvável), o administrador deve ser notificado via log e resolver manualmente ou expandindo o sobrenome fonte.

16.4 Implementação — JavaScript (Node.js/servidor)

/opt/coude-api/username_generator.js

```
"use strict";

// Remove acentos e converte para lowercase ASCII puro
function normalize(str) {
    return str
        .toLowerCase()
        .normalize("NFD")
        .replace(/[\u0300-\u036f]/g, "") // remove diacríticos
```

```

    .replace(/^[^a-z\s]/g, "")          // apenas letras e espaços
    .trim();
}

// Partículas descartadas na composição do sobrenome
const PARTICLES = new Set(["de", "da", "do", "dos", "das", "e", "di", "du", "van", "von"]);

// Extrai [primeiroNome, sobrenomePrincipal, sobrenomeSecundário]
function parseName(fullName) {
    const parts = normalize(fullName).split(/\s+/).filter(Boolean);
    const first = parts[0];
    // Pegar últimos sobrenomes não-partícula
    const surnames = parts.slice(1).filter(p => !PARTICLES.has(p));
    const last = surnames[surnames.length - 1] || parts[1] || first;
    const second = surnames[surnames.length - 2] || null;
    return { first, last, second };
}

// Gera substrings do início, tamanho minLen até maxLen
function prefixFragments(str, minLen = 3, maxLen = 5) {
    const frags = [];
    const limit = Math.min(maxLen, str.length);
    for (let k = minLen; k <= limit; k++) {
        frags.push(str.slice(0, k));
    }
    return frags;
}

// Produto cartesiano de dois arrays
function cartesian(a, b) {
    const result = [];
    for (const x of a)
        for (const y of b)
            result.push(x + y);
    return result;
}

// Gera todos os candidatos para um par (first, surname)
function candidatesFor(first, surname) {
    const nFrag = prefixFragments(first, 3, 5); // thi, thia, thiag
    const sFrag = prefixFragments(surname, 3, 5); // cer, cerq, cerqu

    const forward = cartesian(nFrag, sFrag); // thicer, thiacerq ...
    const backward = cartesian(sFrag, nFrag); // certhi, cerqthi ...

    // Combinar: forward primeiro (mais natural), backward depois
    const all = [...forward, ...backward];

    // Filtrar por comprimento (4-8) e remover duplicatas
    const seen = new Set();
    return all.filter(c => {
        if (c.length < 4 || c.length > 8) return false;
        if (seen.has(c)) return false;
        seen.add(c);
        return true;
    });
}

/**
 * Ponto de entrada principal.
 * @param {string} fullName - Nome completo do usuário
 * @param {Set} existingUsernames - Conjunto de usernames já em uso
 * @returns {string} username único gerado
 */
function generateUsername(fullName, existingUsernames = new Set()) {
    const { first, last, second } = parseName(fullName);

    // Fase 1: combinações com último sobrenome
    let candidates = candidatesFor(first, last);

```

```

for (const c of candidates) {
  if (!existingUsernames.has(c)) return c;
}

// Fase 2: se colidir tudo, usar segundo sobrenome como fonte
if (second) {
  const extra = candidatesFor(first, second);
  for (const c of extra) {
    if (!existingUsernames.has(c)) return c;
  }
}

// Fallback extremo: cruzar ambos os sobrenomes
if (second) {
  const cross = candidatesFor(last, second);
  for (const c of cross) {
    if (!existingUsernames.has(c)) return c;
  }
}

throw new Error(`USERNAME_EXHAUSTED: ${fullName}`);
}

module.exports = { generateUsername, parseName };

```

16.5 Implementação — PHP (para sistemas legados)

UsernameGenerator.php

```

<?php
class UsernameGenerator {

  private static array $PARTICLES =
    ["de", "da", "do", "dos", "das", "e", "di", "du", "van", "von"];

  private static function normalize(string $str): string {
    $str = mb_strtolower($str, "UTF-8");
    $str = iconv("UTF-8", "ASCII//TRANSLIT", $str);
    return preg_replace("/[^a-z\s]/", "", $str);
  }

  private static function parseName(string $fullName): array {
    $parts = array_filter(explode(" ", self::normalize($fullName)));
    $parts = array_values($parts);
    $first = $parts[0] ?? "";
    $surnames = array_filter(
      array_slice($parts, 1),
      fn($p) => !in_array($p, self::$PARTICLES)
    );
    $surnames = array_values($surnames);
    $last = end($surnames) ?: ($parts[1] ?? $first);
    $second = count($surnames) >= 2
      ? $surnames[count($surnames)-2] : null;
    return compact("first", "last", "second");
  }

  private static function prefixFragments(string $s, int $min=3, int $max=5): array {
    $frags = [];
    $len = min($max, strlen($s));
    for ($k = $min; $k <= $len; $k++)
      $frags[] = substr($s, 0, $k);
    return $frags;
  }

  private static function candidatesFor(string $a, string $b): array {

```

```

    $af = self::prefixFragments($a);
    $bf = self::prefixFragments($b);
    $all = [];
    foreach ($af as $x) foreach ($bf as $y) $all[] = $x.$y;
    foreach ($bf as $x) foreach ($af as $y) $all[] = $x.$y;
    $seen = []; $result = [];
    foreach ($all as $c) {
        $l = strlen($c);
        if ($l < 4 || $l > 8 || isset($seen[$c])) continue;
        $seen[$c] = true; $result[] = $c;
    }
    return $result;
}

public static function generate(string $fullName, array $existing = []): string {
    $existSet = array_flip($existing);
    [",first" => $first, "last" => $last, "second" => $second]
    = self::parseName($fullName);
    foreach ([
        self::candidatesFor($first, $last),
        $second ? self::candidatesFor($first, $second) : [],
        $second ? self::candidatesFor($last, $second) : [],
    ] as $batch) {
        foreach ($batch as $c)
            if (!isset($existSet[$c])) return $c;
    }
    throw new RuntimeException("USERNAME_EXHAUSTED: $fullName");
}
}
?>

```

16.6 Tabela de Exemplos

Nome Completo	Primeiro candidato	Candidatos seguintes (se colisão)	Total de variações estimadas
Thiago Cerqueira	thicer	thiacerq, thiagcer, certhi, cerqthia...	~24
Ana Lima	analim	anaali, limaana, limana...	~12
Lucas Oliveira	lucoli	lucaoliv, oliluc, olivluca...	~20
João da Silva	joasil	joaosilv, siljoa, silvjoao...	~18
Maria das Dores	mardor	mariadore, dormari, doremari...	~16
Pedro Alvares Cabral	pedcab	pedralv, cabrped, alvped...	~30

 **DECISÃO TÉCNICA:** A abordagem Combinatorial Substring Generation evita colisões sem números. Para 300 usuários com nomes distintos, a probabilidade de esgotar o espaço de candidatos é estatisticamente desprezível. Em caso extremo, o administrador é notificado via log e pode expandir para uso do segundo sobrenome como fonte primária.

17. API de Integração de Cadastro (Design Completo)

17.1 Decisão Arquitetural: REST API com Webhook de Saída

A integração adota uma abordagem híbrida: REST API síncrona para operações de criação/remoção/restauração (onde o sistema externo precisa da resposta imediata, especialmente o username gerado), combinada com webhook de saída para notificações assíncronas de eventos relevantes (falhas de purge, alertas de disco, expiração de soft delete).

Abordagem	Adequada para COUDE?	Justificativa
REST API pura (síncrona)	Sim (principal)	O sistema externo precisa do username gerado antes de exibir ao usuário — resposta síncrona é obrigatória
Webhook puro (assíncrono)	Não como principal	O sistema externo não saberia o username sem fazer polling subsequente
Híbrido REST + Webhook de saída	Sim (adotado)	REST para operações CRUD, webhook para eventos do servidor de volta ao sistema externo

 **DECISÃO TÉC.:** A decisão pela REST API síncrona como canal principal é motivada por um requisito concreto: o sistema externo da COUDE precisa exibir o username gerado ao usuário no mesmo fluxo de cadastro. Uma abordagem assíncrona criaria uma UX quebrada onde o usuário esperaria por um dado que chegaria depois.

17.2 Estrutura de Endpoints

Método	Endpoint	Responsabilidade	Retorno principal
POST	/api/v1/usuarios/cadastrar	Cadastrar aluno/professor/monitor	201 + username gerado
POST	/api/v1/usuarios/remover	Soft delete (desativar conta)	200 + data de purge
POST	/api/v1/usuarios/restaurar	Reativar dentro dos 30 dias	200 + confirmação
GET	/api/v1/usuarios/:username	Consultar status de uma conta	200 + detalhes
PUT	/api/v1/usuarios/:username/turma	Trocar usuário de turma	200 + confirmação
POST	/api/v1/turmas/criar	Criar nova turma + pasta	201 + path criado
DELETE	/api/v1/turmas/:nome	Arquivar turma encerrada	200 + confirmação
GET	/api/v1/health	Health check do servidor	200 + status

17.3 Contrato da Requisição de Cadastro

POST /api/v1/usuarios/cadastrar — corpo da requisição (JSON):

```
{
  "nome_completo": "Thiago Cerqueira",          // obrigatório
  "email": "thiago@email.com",                  // obrigatório, validado como RFC5321
  "cpf": "123.456.789-00",                      // obrigatório, validado (formato +
dígitos)
  "cargo": "aluno",                             // obrigatório: aluno|professor|
monitor|admin
  "turma": "turma_fullstack_001",               // obrigatório para alunos e monitores
  "senha_hash": "$2y$12$...",                  // obrigatório, bcrypt hash (custo
>=12)
  "metadata": {
    "id_externo": "EXT-2026-0142",              // ID do usuário no sistema externo
(idempotência)
    "matricula": "MAT2026001",                  // opcional
    "data_inicio": "2026-03-01"                // opcional (ISO 8601)
  }
}
```

Resposta de sucesso — 201 Created:

```
{
  "ok": true,
  "username": "thicer",                         // USERNAME GERADO PELO SERVIDOR
  "email_ad": "thicer@ad.coude.com.br",
  "pasta": "/srv/samba/users/thicer",
  "grupos": ["alunos", "turma_fullstack_001"],
  "criado_em": "2026-04-19T14:32:11Z",
  "id_externo": "EXT-2026-0142"
}
```

Resposta de erro — 422 Unprocessable Entity (validação):

```
{
  "ok": false,
  "erro": "VALIDATION_ERROR",
  "detalhes": [
    { "campo": "cpf", "msg": "CPF inválido – dígitos verificadores incorretos" },
    { "campo": "turma", "msg": "Turma turma_xyz_999 não existe no sistema" }
  ],
  "request_id": "req_7f3a2b9c"
}
```

Resposta de erro — 409 Conflict (idempotência):

```
{
  "ok": true,
  "username": "thicer",
  "idempotente": true,
  "msg": "Usuário já existe com id_externo EXT-2026-0142. Retornando dados
existentes."
}
```

17.4 Regras de Validação

Campo	Regra	Comportamento em falha
nome_completo	Mínimo 2 palavras, somente letras + espaços	422 + detalhes
email	RFC 5321, domínio com MX válido	422 + detalhes
cpf	Formato XXX.XXX.XXX-XX + dígitos	422 + detalhes

	verificadores	
cargo	Enum: aluno professor monitor admin	422 + detalhes
turma	Deve existir no AD; obrigatório para aluno/monitor	422 + detalhes
senha_hash	Deve ser bcrypt hash com custo >= 12	422 + detalhes
id_externo	Se enviado e já existir: idempotência — 200+dados	nenhuma falha

17.5 Idempotência

O campo `id_externo` no metadata garante idempotência: se o sistema externo reenviar o mesmo cadastro (retry após timeout), o servidor detecta que o `id_externo` já existe, não recria o usuário, e retorna os dados originais com flag idempotente: `true`. Isso evita usuários duplicados em caso de falhas de rede.

 **SEGURANÇA:** O `id_externo` deve ser único e controlado pelo sistema externo. Nunca aceitar `id_externo` como input de controle de segurança — apenas como chave de idempotência.

17.6 Segurança da API

Mecanismo	Implementação
Autenticação	Header X-API-Key com chave HMAC-SHA256 de 64 hex chars (gerada no servidor)
Autorização	A chave determina o nível de acesso (read-only vs read-write vs admin)
TLS	HTTPS obrigatório em produção (certificado Let's Encrypt ou auto-assinado interno)
Rate Limiting	60 req/min por IP, 200 req/min por API-key; resposta 429 com Retry-After
Input sanitization	Toda entrada validada antes de qualquer chamada a samba-tool
Log de auditoria	Cada operação registrada: timestamp, endpoint, IP, <code>api_key_hash</code> , resultado
Expose mínima	A API escuta apenas em 192.168.1.10:8080 — não exposta para internet (UFW)

Gerar chave de API segura (executar no servidor):

```
python3 -c "import secrets; print(secrets.token_hex(32))"
# Resultado: 64 hex chars – armazenar em /etc/coude/api.env (chmod 600)

# Testar autenticação:
curl -H "X-API-Key: SUA_CHAVE" http://192.168.1.10:8080/api/v1/health
```

17.7 Implementação Completa — Flask + Gerador de Username

/opt/coude-api/app.py (versão v2):

```
#!/usr/bin/env python3
```

```
"""COUDE API v2 – Cadastro, Username Generation, Soft Delete"""

import os, subprocess, logging, hashlib, re, uuid
from datetime import datetime
from flask import Flask, request, jsonify, g
from functools import wraps

app = Flask(__name__)

logging.basicConfig(
    filename="/var/log/coude/api.log",
    level=logging.INFO,
    format="%(asctime)s %(levelname)s [%(request_id)s] %(message)s"
)

API_KEY = os.environ.get("COUDE_API_KEY", "")
RATE_LIMIT_STORE = {} # produção: usar Redis

# — Decoradores —————

def require_auth(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        key = request.headers.get("X-API-Key", "")
        if not key or key != API_KEY:
            return jsonify({"ok": False, "erro": "UNAUTHORIZED"}), 401
        return f(*args, **kwargs)
    return decorated

def attach_request_id(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        g.request_id = "req_" + uuid.uuid4().hex[:8]
        return f(*args, **kwargs)
    return decorated

# — Validação —————

def validate_cpf(cpf: str) -> bool:
    digits = re.sub(r"\D", "", cpf)
    if len(digits) != 11 or len(set(digits)) == 1: return False
    def calc(d, n):
        s = sum(int(d[i])*(n-i) for i in range(n-1))
        r = (s*10)%11
        return 0 if r >= 10 else r
    return calc(digits,10)==int(digits[9]) and calc(digits,11)==int(digits[10])

def validate_request(data: dict) -> list:
    errors = []
    name = data.get("nome_completo", "").strip()
    if len(name.split()) < 2:
        errors.append({"campo": "nome_completo", "msg": "Mínimo de 2 palavras"})
    if not re.match(r"^[^@]+@[^@]+\.[^@]+$", data.get("email", "")):
        errors.append({"campo": "email", "msg": "Email inválido"})
    if not validate_cpf(data.get("cpf", "")):
        errors.append({"campo": "cpf", "msg": "CPF inválido"})
    if data.get("cargo") not in ["aluno", "professor", "monitor", "admin"]:
        errors.append({"campo": "cargo", "msg": "Cargo inválido"})
    if data.get("cargo") in ["aluno", "monitor"] and not data.get("turma"):
        errors.append({"campo": "turma", "msg": "Obrigatório para este cargo"})
    if not data.get("senha_hash", "").startswith("$2"):
        errors.append({"campo": "senha_hash", "msg": "Deve ser bcrypt hash"})
    return errors
```

```

# — Geração de username (chama o módulo JS via subprocess)
# Em produção, implementar a lógica diretamente em Python
def generate_username(full_name: str, existing: set) -> str:
    """
    Implementação Python do algoritmo Combinatorial Substring Generation.
    Espelha identicamente a lógica do username_generator.js.
    """
    import unicodedata
    def norm(s):
        s = s.lower()
        s = unicodedata.normalize("NFD", s)
        s = "".join(c for c in s if unicodedata.category(c) != "Mn")
        return re.sub(r"^[a-z\s]", "", s).strip()

    PARTICLES = {"de", "da", "do", "dos", "das", "e", "di", "du", "van", "von"}
    parts = norm(full_name).split()
    first = parts[0]
    surnames = [p for p in parts[1:] if p not in PARTICLES]
    last = surnames[-1] if surnames else (parts[1] if len(parts)>1 else first)
    second = surnames[-2] if len(surnames)>=2 else None

    def frags(s, mn=3, mx=5):
        return [s[:k] for k in range(mn, min(mx, len(s))+1)]

    def candidates_for(a, b):
        af, bf = frags(a), frags(b)
        all_c = [x+y for x in af for y in bf]
        all_c += [x+y for x in bf for y in af]
        seen, result = set(), []
        for c in all_c:
            if 4 <= len(c) <= 8 and c not in seen:
                seen.add(c); result.append(c)
        return result

    for batch in filter(None, [
        candidates_for(first, last),
        candidates_for(first, second) if second else None,
        candidates_for(last, second) if second else None,
    ]):
        for c in batch:
            if c not in existing:
                return c

    raise ValueError(f"USERNAME_EXHAUSTED: {full_name}")

def get_existing_usernames() -> set:
    result = subprocess.run(
        ["samba-tool", "user", "list"], capture_output=True, text=True
    )
    return set(result.stdout.strip().splitlines())

# — Endpoints —————

@app.route("/api/v1/health")
def health():
    return jsonify({"status": "ok", "servidor": "srv1.ad.coude.com.br", "versao": "2.0"})

@app.route("/api/v1/usuarios/cadastrar", methods=["POST"])
@require_auth
@attach_request_id
def cadastrar():
    data = request.get_json() or {}

```

```

# Idempotência: checar id_externo
id_ext = (data.get("metadata") or {}).get("id_externo")
if id_ext:
    check = subprocess.run(
        ["samba-tool", "user", "show", f"--filter=description={id_ext}"],
        capture_output=True, text=True
    )
    if check.returncode == 0:
        username = check.stdout.split("sAMAccountName:")[1].split()[0]
        return jsonify({"ok": True, "username": username, "idempotente": True}), 200

# Validação
errors = validate_request(data)
if errors:
    return jsonify({"ok": False, "erro": "VALIDATION_ERROR", "detalhes": errors,
                    "request_id": g.request_id}), 422

# Gerar username
try:
    existing = get_existing_usernames()
    username = generate_username(data["nome_completo"], existing)
except ValueError as e:
    logging.error(str(e))
    return jsonify({"ok": False, "erro": "USERNAME_EXHAUSTED"}), 500

# Criar no AD
cargo = data["cargo"]
turma = data.get("turma", "")
ou_map = {
    "aluno": "OU=Alunos, OU=Usuarios, DC=ad, DC=coude, DC=com, DC=br",
    "professor": "OU=Professores, OU=Usuarios, DC=ad, DC=coude, DC=com, DC=br",
    "monitor": "OU=Monitores, OU=Usuarios, DC=ad, DC=coude, DC=com, DC=br",
    "admin": "OU=Administradores, OU=Usuarios, DC=ad, DC=coude, DC=com, DC=br",
}
create = subprocess.run([
    "samba-tool", "user", "create", username, "--random-password",
    f"--given-name={data['nome_completo']}",
    f"--userou={ou_map[cargo]}",
    f"--mail-address={username}@ad.coude.com.br",
    f"--description={id_ext or ''}",
], capture_output=True, text=True)

if create.returncode != 0:
    return jsonify({"ok": False, "erro": create.stderr, "request_id": g.request_id}),
500

# Adicionar a grupos
group_map = {"aluno": "alunos", "professor": "professores",
             "monitor": "monitores", "admin": "administradores"}
subprocess.run(["samba-tool", "group", "addmembers", group_map[cargo], username])
if turma:
    subprocess.run(["samba-tool", "group", "addmembers", turma, username])

# Criar pasta no file server
cargo_dir =
{"aluno": "aluno", "professor": "prof", "monitor": "monitor", "admin": "admin"}
subprocess.run(["/usr/local/bin/coude-create-user-
dir.sh", username, cargo_dir[cargo]])

logging.info(f"CADASTRO|{username}|{cargo}|{turma}|{g.request_id}")
return jsonify({
    "ok": True,
    "username": username,
    "email_ad": f"{username}@ad.coude.com.br",
    "pasta": f"/srv/samba/users/{username}",
    "grupos": [group_map[cargo]] + ([turma] if turma else []),
    "criado_em": datetime.utcnow().isoformat()+"Z",
    "request_id": g.request_id
}), 201

```

```

@app.route("/api/v1/usuarios/remove", methods=["POST"])
@require_auth
def remover():
    data = request.get_json() or {}
    username= data.get("username","").strip()
    motivo = data.get("motivo","nao_informado")
    if not username:
        return jsonify({"ok":False,"erro":"username obrigatório"}), 400
    result = subprocess.run(["/usr/local/bin/coude-soft-delete.sh",username],
                           capture_output=True, text=True)
    if result.returncode != 0:
        return jsonify({"ok":False,"erro":result.stderr}), 500
    return jsonify({"ok":True,"username":username,
                  "exclusao_definitiva_em":"30 dias","motivo":motivo})

@app.route("/api/v1/usuarios/restaurar", methods=["POST"])
@require_auth
def restaurar():
    data = request.get_json() or {}
    username= data.get("username","").strip()
    result = subprocess.run(["samba-tool","user","enable",username],
                           capture_output=True, text=True)
    if result.returncode != 0:
        return jsonify({"ok":False,"erro":result.stderr}), 500
    return jsonify({"ok":True,"username":username})

@app.route("/api/v1/turmas/criar", methods=["POST"])
@require_auth
def criar_turma():
    data = request.get_json() or {}
    turma = data.get("nome","").strip()
    if not re.match(r"^turma_[a-z]_[0-9]{3}$", turma):
        return jsonify({"ok":False,"erro":"Nome inválido. Padrão:
turma_[curso]_[número 3 dígitos]"}), 422
    subprocess.run(["samba-tool","group","add",turma])
    result = subprocess.run(["/usr/local/bin/coude-create-turma-dir.sh",turma],
                           capture_output=True, text=True)
    return jsonify({"ok":True,"turma":turma,
                  "pasta":f"/srv/samba/turmas/{turma}"}), 201

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080)

```

17.8 Webhook de Saída (eventos do servidor → sistema externo)

Para eventos assíncronos (conta purgada, disco chegando a 80%, falha de backup), o servidor envia webhooks para um endpoint configurado no sistema externo da escola.

Configuração do webhook em `/etc/coude/webhook.env`:

```

COUDE_WEBHOOK_URL=https://sistema.coude.com.br/webhooks/servidor
COUDE_WEBHOOK_SECRET=HMAC_SHA256_SECRET_AQUI

```

Função de envio de webhook:

```

import hmac, hashlib, requests, json, os

def send_webhook(event: str, payload: dict):
    url = os.getenv("COUDE_WEBHOOK_URL")
    secret = os.getenv("COUDE_WEBHOOK_SECRET","").encode()
    if not url: return

```

```
body = json.dumps({"event": event, "data": payload, "ts":
datetime.utcnow().isoformat()})
sig = hmac.new(secret, body.encode(), hashlib.sha256).hexdigest()
try:
    requests.post(url, data=body,
        headers={"Content-Type": "application/json", "X-Coude-Signature": sig},
        timeout=5)
except Exception as e:
    logging.warning(f"WEBHOOK_FAILED|{event}|{e}")

# Exemplos de uso:
# send_webhook("user.purged", {"username": "thicer", "data": "2026-05-20"})
# send_webhook("disk.warning", {"uso_pct": 82, "path": "/srv/samba"})
```

Evento de Webhook	Quando é disparado
user.created	Usuário criado com sucesso — inclui username gerado
user.soft_deleted	Conta desativada — inclui data de purge
user.purged	Conta definitivamente removida após 30 dias
user.restored	Conta reativada dentro do prazo
disk.warning	Disco de /srv/samba atingiu 80% de uso
backup.failed	Job de backup rsync falhou
ad.error	Samba AD retornou erro em operação

18. Modelo de Permissões e Armazenamento (RBAC)

18.1 Justificativa do Modelo: RBAC com Hierarquia de Cargos

O modelo adotado é Role-Based Access Control (RBAC) com hierarquia de cargos (student → monitor → professor → admin) e isolamento por turma. A alternativa ACL pura por usuário foi descartada por ser impossível de manter com 300 usuários — a adição de um único aluno exigiria atualizar ACLs em dezenas de pastas. Com RBAC, a atribuição do usuário a um grupo propaga automaticamente as permissões corretas.

18.2 Hierarquia de Cargos

Hierarquia de permissões (menor → maior):

aluno < monitor < professor < administrador

Regra de herança:

- Cada nível inclui as permissões do nível anterior
- Um professor tem tudo que um monitor tem + mais
- Um admin tem tudo que um professor tem + mais

Cargo	Pasta pessoal	Pasta da turma	Pasta de outros alunos
Aluno	Full control	Leitura em _geral; escrita na própria subpasta	Nenhum acesso
Monitor	Full control	Leitura em _geral e nas subpastas de alunos	Leitura
Professor	Full control	Full control em toda a pasta da turma	Leitura + escrita + exclusão
Administrador	Full control em todas	Full control em todas as turmas	Full control em tudo

18.3 Modelo de Pastas por Turma

Estrutura detalhada com permissões anotadas:

```

/srv/samba/turmas/turma_fullstack_001/
├── _geral/
│   ├── Materiais/
│   └── Avisos/
└── alunos/
    ├── thicer/
    │   ├── Projetos/
    │   └── Entregas/
    └── joasil/

```

← pasta compartilhada da turma
→ Professor: rwx | Monitor: r-x | Aluno: r-x
→ Professor: rwx | Monitor: r-x | Aluno: r-x

← container de subpastas individuais
→ thicer: rwx | Professor: rwx | Monitor: r-x
→ thicer: rwx | Professor: rwx | Monitor: r-x
→ thicer: rwx | Professor: rwx | Monitor: r-x
→ joasil: rwx | Professor: rwx | Monitor: r-x

18.3.1 Decisão: Alunos têm leitura na pasta _geral (não escrita)

 **DECISÃO TÉCNICA:** Alunos têm acesso de LEITURA na pasta _geral da turma, mas não podem escrever. A razão é preservar a integridade dos materiais didáticos: um aluno não deve poder sobrescrever ou apagar um arquivo de aula postado pelo professor. Se precisar enviar material para a turma, o aluno usa sua própria subpasta em alunos/. Professores e monitores têm acesso total à _geral.

18.3.2 Decisão: Professores podem editar/excluir na pasta do aluno

 **DECISÃO TÉCNICA:** Professores têm Full Control na pasta do aluno por duas razões pedagógicas: (1) podem corrigir, anotar e devolver arquivos de trabalhos; (2) podem remover arquivos impróprios ou problemáticos sem precisar de intervenção do admin. Essa decisão implica registrar em log qualquer operação destrutiva do professor sobre pasta de aluno — ver item 18.5.

18.3.3 Decisão: Monitores têm leitura (não escrita) nas pastas de alunos

 **DECISÃO TÉCNICA:** Monitores podem visualizar o trabalho dos alunos para fins de suporte, mas não têm permissão de escrita nas pastas individuais. Isso evita conflito de responsabilidade com o professor e protege o aluno de alterações não autorizadas em seus arquivos. Monitores **SÍ** têm escrita na pasta `_geral` da turma para postar materiais de apoio.

18.4 Script: Configurar Permissões de Turma (RBAC via ACL POSIX)

`/usr/local/bin/coude-create-turma-dir.sh` (v2 com RBAC):

```
#!/bin/bash
# Uso: coude-create-turma-dir.sh <nome_turma>
TURMA=$1
BASE="/srv/samba/turmas/$TURMA"

# Criar estrutura
mkdir -p "$BASE/_geral"/{Materiais,Avisos}
mkdir -p "$BASE/alunos"

# — Pasta _geral —————
chown root:"Domain Admins" "$BASE/_geral"
chmod 750 "$BASE/_geral"

# Professores: full control (rwx)
setfacl -R -m g:professores:rwx "$BASE/_geral"
setfacl -R -d -m g:professores:rwx "$BASE/_geral"

# Monitores: leitura/execução apenas (r-x)
setfacl -R -m g:monitores:r-x "$BASE/_geral"
setfacl -R -d -m g:monitores:r-x "$BASE/_geral"

# Alunos da turma: leitura/execução (r-x)
setfacl -R -m g:"$TURMA":r-x "$BASE/_geral"
setfacl -R -d -m g:"$TURMA":r-x "$BASE/_geral"

# Admins: full control
setfacl -R -m g:"Domain Admins":rwx "$BASE/_geral"
setfacl -R -d -m g:"Domain Admins":rwx "$BASE/_geral"

# — Pasta alunos/ (container) —————
chown root:"Domain Admins" "$BASE/alunos"
chmod 750 "$BASE/alunos"

# Professores: full control para poder listar e acessar
setfacl -m g:professores:rwx "$BASE/alunos"
setfacl -d -m g:professores:rwx "$BASE/alunos"

# Monitores: listagem e acesso (r-x)
setfacl -m g:monitores:r-x "$BASE/alunos"
setfacl -d -m g:monitores:r-x "$BASE/alunos"

echo "Turma $TURMA criada com modelo RBAC completo."
```

/usr/local/bin/coude-add-aluno-turma.sh (criar pasta individual do aluno):

```
#!/bin/bash
# Chamado após adicionar aluno a uma turma
# Uso: coude-add-aluno-turma.sh <username> <turma>
USERNAME=$1; TURMA=$2
ALUNODIR="/srv/samba/turmas/$TURMA/alunos/$USERNAME"

mkdir -p "$ALUNODIR"/{Projetos,Entregas}

# Dono: o próprio aluno, grupo: turma
chown "$USERNAME:$TURMA" "$ALUNODIR"
chmod 700 "$ALUNODIR"

# Aluno: full control da própria pasta
setfacl -R -m u:"$USERNAME":rwx "$ALUNODIR"
setfacl -R -d -m u:"$USERNAME":rwx "$ALUNODIR"

# Professores: full control (ler, corrigir, devolver)
setfacl -R -m g:professores:rwx "$ALUNODIR"
setfacl -R -d -m g:professores:rwx "$ALUNODIR"

# Monitores: leitura (suporte, sem escrita)
setfacl -R -m g:monitores:r-x "$ALUNODIR"
setfacl -R -d -m g:monitores:r-x "$ALUNODIR"

# Outros alunos: nenhum acesso (padrão do chmod 700)

# Admins: full control
setfacl -R -m g:"Domain Admins":rwx "$ALUNODIR"
setfacl -R -d -m g:"Domain Admins":rwx "$ALUNODIR"

echo "Pasta de $USERNAME em $TURMA configurada (RBAC)."
```

18.5 Auditoria de Operações Destrutivas

Como professores têm Full Control na pasta de alunos, toda operação de exclusão deve ser registrada. O Samba pode ser configurado com o módulo `full_audit` para isso:

Adicionar ao `smb.conf`, seção `[turmas]`:

```
vfs objects = acl_xattr full_audit
full_audit:prefix = %u|%I|%T
full_audit:success = unlink rmdir rename write
full_audit:failure = none
full_audit:facility = LOCAL7
full_audit:priority = NOTICE
```

Configurar `rsyslog` para capturar o audit:

```
# /etc/rsyslog.d/oude-audit.conf
local7.notice /var/log/oude/file_audit.log
```



SEGURANÇA: O log de auditoria permite rastrear quem deletou ou renomeou qualquer arquivo nas pastas de turma. Em caso de disputas pedagógicas, o admin pode consultar `/var/log/oude/file_audit.log` para determinar a origem de uma alteração.

18.6 Escalabilidade do Modelo RBAC

Cenário futuro	Impacto no modelo	Ação necessária
----------------	-------------------	-----------------

Nova turma criada	Zero impacto	Executar coude-create-turma-dir.sh
Novo cargo: supervisor	Baixo	Criar grupo, definir nível na hierarquia, ajustar scripts
Aluno muda de turma	Baixo	Remover do grupo da turma antiga, adicionar à nova, mover pasta
500+ usuários	Baixo (ACL POSIX escala)	Monitorar performance do Samba em disco; considerar separar FS
Múltiplos laboratórios	Baixo	Adicionar OU por laboratório, replicar modelo de turmas
Acesso remoto (VPN)	Médio	WireGuard + autenticação AD, mesmas permissões se aplicam

19. Melhorias de Engenharia (Pensamento de Produção)

19.1 Escalabilidade

19.1.1 Separação de serviços quando necessário

A arquitetura atual (DC + FS + API no mesmo servidor) é adequada até aproximadamente 500 usuários simultâneos ou 150 GB de tráfego diário de SMB. Quando o crescimento se aproximar desses limites, o caminho de evolução é:

- Fase 1 atual: 1 servidor — Samba AD + FS + API (Intel NUC)
- Fase 2: separar FS em um segundo servidor (NAS ou segundo NUC)
- Fase 3: adicionar segundo DC (dc2.ad.coude.com.br) para alta disponibilidade
- Fase 4: API em contêiner Docker/Podman separado com Redis para rate limiting e cache

Indicadores que sinalizam necessidade de escalar:

```
# Latência de login > 3 segundos
# smbstatus | wc -l > 50 conexões simultâneas
# df -h /srv/samba → uso > 70%
# top → CPU do samba-ad-dc > 60% em horário de aula
```

19.1.2 Quotas de disco por usuário

Configurar quota POSIX para evitar que um usuário consuma todo o disco:

```
apt install quota -y

# Ativar quotas no fstab para /srv
# UUID=... /srv ext4 defaults,acl,usrquota,grpquota 0 2
mount -o remount /srv
quotacheck -cum /srv
quotaon /srv

# Definir quota por usuário (ex: 5 GB soft, 6 GB hard)
setquota -u thicer 5G 6G 0 0 /srv

# Script para aplicar quota a todos os alunos:
for user in $(samba-tool group listmembers alunos); do
    setquota -u "$user" 5368709120 6442450944 0 0 /srv
done
```

19.2 Segurança

19.2.1 Hardening do SSH

/etc/ssh/sshd_config — configurações obrigatórias:

```
PermitRootLogin no
PasswordAuthentication no          # usar apenas chave SSH
PubkeyAuthentication yes
AuthorizedKeysFile .ssh/authorized_keys
AllowUsers admin.thiago           # somente admin acessa por SSH
MaxAuthTries 3
LoginGraceTime 30
ClientAliveInterval 300
ClientAliveCountMax 2
```

19.2.2 Rotação automática da chave de API

Script de rotação (executar periodicamente ou via CI/CD):

```
#!/bin/bash
# /usr/local/bin/coude-rotate-api-key.sh
NOVA_CHAVE=$(python3 -c "import secrets; print(secrets.token_hex(32))")

# Atualizar arquivo de configuração
sed -i "s/COUDE_API_KEY=.* /COUDE_API_KEY=$NOVA_CHAVE/" /etc/coude/api.env

# Reiniciar API com nova chave
systemctl restart coude-api

# Notificar admin
echo "[COUDE] Nova API key gerada: $(date)" >> /var/log/coude/security.log
echo "Enviar nova chave ao sistema externo: $NOVA_CHAVE"

# IMPORTANTE: após rodar, comunicar a nova chave ao sistema externo da escola.
```

19.2.3 Política de senhas no AD

Definir política de senhas fine-grained para alunos vs admin:

```
# Política para Domain (padrão – alunos e professores)
samba-tool domain passwordsettings set \
  --complexity=on \
  --history-length=5 \
  --min-pwd-length=10 \
  --max-pwd-age=90 \
  --min-pwd-age=1

# Política mais rígida para administradores (fine-grained PSO)
samba-tool domain passwordsettings pso create admin-policy 10 \
  --complexity=on \
  --history-length=10 \
  --min-pwd-length=16 \
  --max-pwd-age=30

samba-tool domain passwordsettings pso apply admin-policy administradores
```

19.2.4 Certificado TLS para a API

Em produção, a API deve obrigatoriamente usar HTTPS, mesmo em rede interna. Um certificado auto-assinado interno é suficiente já que apenas o sistema externo da escola consome a API.

Gerar certificado auto-assinado (valido por 2 anos):

```
openssl req -x509 -nodes -days 730 \
  -newkey rsa:4096 \
  -keyout /etc/coude/api.key \
  -out /etc/coude/api.crt \
  -subj "/CN=srv1.ad.coude.com.br/O=COUDE/C=BR"

chmod 600 /etc/coude/api.key

# Atualizar gunicorn para usar TLS:
# ExecStart=...gunicorn ... --certfile=/etc/coude/api.crt
--keyfile=/etc/coude/api.key
```

19.3 Manutenibilidade

19.3.1 Documentação viva no servidor

Criar página de status interno acessível na rede:

```
# Adicionar endpoint /api/v1/status que retorna JSON com:
# - versão da API
# - uptime do samba-ad-dc
# - uso de disco
# - total de usuários por cargo
# - últimas 5 operações no log

# Exemplo de resposta:
{
  "versao": "2.0",
  "samba_status": "running",
  "disco_uso_pct": 45,
  "usuarios": { "alunos": 287, "professores": 12, "monitores": 8 },
  "ultima_operacao": "2026-04-19T14:32:11Z"
}
```

19.3.2 Versionamento dos scripts

Todos os scripts em `/usr/local/bin/coude-*.sh` devem ser versionados em um repositório Git interno. Isso permite auditar quem alterou um script, reverter alterações e fazer review antes de aplicar em produção.

Inicializar repositório Git dos scripts:

```
cd /usr/local/bin
git init coude-scripts
cp coude-*.sh coude-scripts/
cd coude-scripts
git add . && git commit -m "chore: versão inicial dos scripts COUDE v2.0"

# Workflow: toda alteração em script deve passar por:
# 1. git checkout -b fix/nome-do-fix
# 2. Editar script
# 3. Testar em VM
# 4. git commit + git merge para main
# 5. Aplicar no servidor de produção
```

19.3.3 Monitoramento com script de health check

`/usr/local/bin/coude-healthcheck.sh` (executar a cada 5 minutos via cron):

```
#!/bin/bash
LOG="/var/log/coude/healthcheck.log"
TS=$(date +%Y-%m-%dT%H:%M:%S)
ERROS=0

check() { local desc=$1; local cmd=$2
  if ! eval "$cmd" > /dev/null 2>&1; then
    echo "$TS FAIL $desc" >> "$LOG"; ERROS=$((ERROS+1))
  fi
}

check "samba-ad-dc" "systemctl is-active samba-ad-dc"
check "coude-api" "systemctl is-active coude-api"
check "dns-local" "host -t A srv1.ad.coude.com.br 127.0.0.1"
check "kerberos" "klist -s"
```

```

check "disco-dados" "[ $(df /srv/samba | awk 'NR==2{print $5}' | tr -d %) -lt 85 ]"
check "api-responde" "curl -sf http://localhost:8080/api/v1/health"

if [ $ERROS -gt 0 ]; then
    echo "$TS ALERT $ERROS verificações falharam" >> "$LOG"
fi

```

19.4 Integridade dos Dados

19.4.1 Estratégia de backup com retenção

Backup diário com retenção de 7 dias por nomenclatura de data:

```

#!/bin/bash
# /usr/local/bin/coude-backup.sh
DEST="/mnt/backup/samba"
DATE=$(date +%Y-%m-%d)
KEEP_DAYS=7

# Criar backup incremental
rsync -av --delete \
    --link-dest="$DEST/latest" \
    /srv/samba/ "$DEST/$DATE/"

# Atualizar symlink "latest"
ln -sf "$DEST/$DATE" "$DEST/latest"

# Remover backups mais antigos que KEEP_DAYS
find "$DEST" -maxdepth 1 -type d -name "2*" \
    -mtime +$KEEP_DAYS -exec rm -rf {} \;

# Backup do AD
samba-tool domain backup online \
    --targetdir="/mnt/backup/ad/$DATE" \
    -U Administrator --password="$SAMBA_ADMIN_PASS"

echo "$DATE|BACKUP_OK" >> /var/log/coude/backup.log

```

19.4.2 Verificação de integridade do AD

Executar semanalmente (cron domingo 03:00):

```

# Verificar replicação e saúde do AD
samba-tool dbcheck --cross-ncs --fix

# Verificar DNS
samba_dnsupdate --verbose --all-names

# Verificar Kerberos
samba-tool testparm

```

19.5 Tabela de Melhorias Priorizadas

Prioridade	Melhoria	Impacto	Esforço
P1 — Imediato	Quotas de disco por usuário	Evita que um usuário quebre o servidor	Baixo (1h)

P1 — Imediato	Certificado TLS na API	Segurança básica de produção	Baixo (30min)
P1 — Imediato	SSH com chave pública + root desabilitado	Reduz superfície de ataque drasticamente	Baixo (30min)
P2 — Semana 1	Política de senhas no AD	Proteção de contas de alunos	Baixo (1h)
P2 — Semana 1	Auditoria full_audit no Samba	Rastreabilidade de operações destrutivas	Baixo (1h)
P2 — Semana 1	Health check em cron (5 min)	Detecção precoce de falhas	Baixo (1h)
P3 — Mês 1	Backup incremental com retenção	Integridade e recuperação de dados	Médio (3h)
P3 — Mês 1	Versionamento Git dos scripts	Manutenibilidade e auditoria	Médio (2h)
P3 — Mês 1	Rotação de chave de API	Segurança da integração	Médio (2h)
P4 — Futuro	Segundo DC (dc2.ad.coude.com.br)	Alta disponibilidade	Alto
P4 — Futuro	Redis para rate limiting da API	Escalabilidade da API	Alto
P4 — Futuro	Dashboard de gestão web (FreelIPA)	Administração por não-técnicos	Alto

20. Checklist Consolidado v2.0

20.1 Servidor — Infraestrutura Base

Item	Status esperado
Debian 12 instalado sem GUI	hostname -f retorna srv1.ad.coude.com.br
IP fixo configurado	ip a mostra 192.168.1.10 estável
/etc/hosts com IP real (não 127.x)	ping srv1.ad.coude.com.br → IP real
Samba AD provisionado e rodando	samba-tool domain level show sem erro
DNS interno funcionando	host -t SRV _ldap._tcp.ad.coude.com.br responde
Kerberos autenticando	kinit Administrator sem erro
OUs criadas	samba-tool ou list mostra todas as OUs
Grupos base criados	samba-tool group list mostra os 4 grupos + turmas
Pastas criadas com ACL POSIX	getfacl /srv/samba/turmas mostra entradas corretas
smb.conf com full_audit ativado	smbtree sem erro + log de audit gerando
Quotas POSIX ativadas por usuário	quota -u thicer retorna limites corretos
SSH com chave pública + root desabilitado	ssh root@srv1 → deve ser recusado
Certificado TLS na API	curl https://srv1:8080/api/v1/health → 200
Firewall UFW ativo com regras corretas	ufw status mostra todas as portas
Cron jobs configurados	crontab -l mostra purge, backup, healthcheck
Health check rodando	/var/log/coude/healthcheck.log sem FAIL
Backup testado	ls /mnt/backup/samba/latest/ mostra dados

20.2 Sistema de Usernames e API

Item	Verificação
Gerador de username retorna resultado legível	generateUsername("Thiago Cerqueira") → "thicer"
Colisão tratada sem números	Criar 2 usuários com mesmo nome → usernames distintos
API responde /health	curl → {"status":"ok"}
Cadastro retorna username gerado	POST /cadastrar → {"username":"thicer"}
Idempotência funciona	POST duplicado com mesmo id_externo → 200 sem duplicar
Validação de CPF rejeita inválido	POST com CPF errado → 422 com campo "cpf"
Soft delete desativa conta no AD	POST /remover → samba-tool user show mostra disabled
Restauração reativa conta	POST /restaurar → conta habilitada novamente
Purge após 30 dias arquiva pasta	Simular data no log → pasta movida para arquivo_morto
Webhook disparado em evento de	Endpoint externo recebe POST com evento user.purged

purge	
-------	--

20.3 Modelo RBAC — Testes de Permissão

Cenário de teste	Resultado esperado	Verificar via
Aluno acessa sua pasta pessoal	Sucesso (leitura/escrita)	Windows Explorer ou smbclient
Aluno acessa pasta de outro aluno	Acesso negado (Erro 5)	smbclient -c "ls" com usuário
Aluno lê _geral da turma	Sucesso (somente leitura)	Tentar criar arquivo → negado
Aluno escreve em _geral da turma	Acesso negado	Tentar criar arquivo
Professor escreve em _geral da turma	Sucesso	Criar arquivo + verificar
Professor acessa pasta de aluno qualquer	Sucesso (full control)	smbclient com usuário prof
Monitor lê pasta de aluno	Sucesso (somente leitura)	smbclient com usuário monitor
Monitor escreve em pasta de aluno	Acesso negado	Tentar criar arquivo
Admin acessa qualquer pasta	Sucesso em tudo	Testar com admin.thiago
Aluno de turma A acessa pasta da turma B	Acesso negado	smbclient de outro usuário